B22 Evaluating LLM systems

Retrieval metrics, answer checks, LLM judges and regression gates for prompt changes.

B22-L1 Evaluate retrieval, recall at k and MRR on a gold set

Score retrieval apart from generation, on a versioned gold set, with recall at k and MRR, and compare settings on the same questions.

```python
def recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float:
    return len(set(ranked[:k]) & relevant) / len(relevant)


def reciprocal_rank(ranked: list[str], relevant: set[str]) -> float:
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0
```

Recall at k	len(set(ranked[:k]) & relevant) / len(relevant). With one relevant section per question, its mean is the hit rate.
Mean reciprocal rank	Mean over questions of 1 / rank of the first relevant result, ranks from 1, or 0 when none appears.
Gold set rules	Real questions, relevance marked by section, versioned, and every failed question added.
Repeated sections in a ranking	list(dict.fromkeys(section_ids)) keeps each at its first position.

B22-L2 Evaluate answers, exact checks, token F1 and LLM judges

Exact checks first, token F1 for trends, and a validated, structured judge for free text, run in both orders for pairs.

Order of answer checks	Exact checks first, then token F1 for trends, then a validated LLM judge for free text.
Token F1	Shared words with Counter &; p = shared / reply words, r = shared / reference words, F1 = 2pr / (p + r).
LLM judge biases (Zheng et al., NeurIPS 2023)	Position, verbosity and self-enhancement bias. Swap the order for pairs and call disagreements ties.
Before trusting a judge	Compare it with human grades on a sample, criterion by criterion, and re-check after any prompt or model change.

B22-L3 Regression evals and release gates for prompt changes

Compare per item, repeat noisy items, write the gate first, and never let a higher average hide a broken critical item.

```python
fixed = sorted(q for q in baseline if not baseline[q] and candidate[q])
broken = sorted(q for q in baseline if baseline[q] and not candidate[q])
```

Fixed and broken items	Fixed: fail in the baseline, pass in the candidate. Broken: the reverse. Read every broken item.
Majority over repeated runs	Pass when sum(runs) > len(runs) / 2. Unstable when 0 < sum(runs) < len(runs).
A release gate	Written before the run: every critical item passes, at most N items break, and the pass rate drops by no more than a set share.

What to log for each eval run	Model, prompt version, eval set version, verdicts per item and token counts.